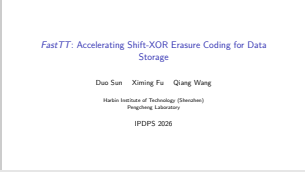
FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

Duo Sun Ximing Fu Qiang Wang

Harbin Institute of Technology (Shenzhen)
Pengcheng Laboratory

IPDPS 2026

FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage



Good morning/afternoon everyone. I am presenting our work, FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage. This talk is about how we turn the state-of-the-art Two-tone Shift-XOR code from a theoretically appealing idea into a practical high-performance implementation for storage systems.

Motivation: Why This Problem Matters

- In distributed storage reliability, two mainstream choices are replication and erasure coding; erasure coding saves substantial storage, and RS codes have been widely deployed with strong theory and system optimizations.
- Two-tone Shift-XOR has excellent theoretical complexity: it uses shift/XOR computations instead of finite-field matrix multiplications.
- But prior Two-tone studies mainly emphasize coding theory, with limited system-level optimization for memory access behavior.
- RS-oriented optimization techniques cannot be directly transferred to Shift-XOR codes because both computation principles and data-access patterns differ.
- We therefore need a dedicated implementation methodology for Two-tone Shift-XOR and practical validation in production-like systems such as Ceph.

Talk Goal

Build system-level, memory-access-aware optimizations for Two-tone Shift-XOR and verify real performance gains in Ceph.

Motivation: Why This Problem Matters
- In distributed storage reliability, two mainstream choices are replication and erasure coding; erasure coding saves substantial storage, and RS codes have been widely deployed with strong theory and system optimizations.
- Two-tone Shift-XOR has excellent theoretical complexity: it uses shift/XOR computations instead of finite-field matrix multiplications.
- But prior Two-tone studies mainly emphasize coding theory, with limited system-level optimization for memory access behavior.
- RS-oriented optimization techniques cannot be directly transferred to Shift-XOR codes because both computation principles and data-access patterns differ.
- We therefore need a dedicated implementation methodology for Two-tone Shift-XOR and practical validation in production-like systems such as Ceph.

Talk Goal
Build system-level, memory-access-aware optimizations for Two-tone Shift-XOR and verify real performance gains in Ceph.

In distributed storage, reliability is usually provided by replication or erasure coding. Replication is simple, while erasure coding is much more storage-efficient. In practice, Reed-Solomon is already widely used and has strong theoretical and system-level optimizations, so our work is not about dismissing RS. Our focus is Two-tone Shift-XOR, which has excellent theoretical complexity because it replaces finite-field matrix multiplications with shifts and XORs. The gap is implementation: prior Two-tone work provides limited system-level optimization for memory access. RS-specific optimization tricks cannot be directly reused because the computation principles and data-access patterns are different. So our goal is to build Shift-XOR-specific implementation optimizations and validate them in Ceph.

Limitations of Existing Solutions (1/2)

- **Redundant Memory Access and Poor Temporal Locality:** Naive Two-tone encoding/decoding repeatedly updates parity locations without memory-friendly scheduling.
- **Inefficient Data Traversal Pattern of Encoding:** Horizontal traversal has better data locality but repeated parity updates; vertical traversal reduces parity updates but increases data reloads.

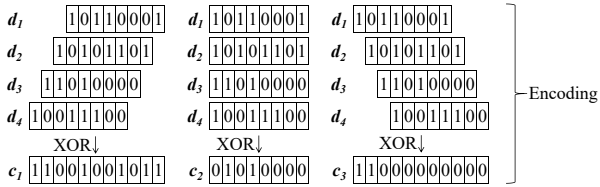
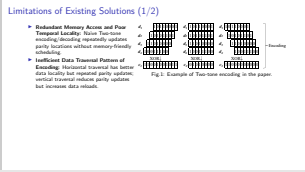


Fig.1: Example of Two-tone encoding in the paper.



This slide covers the first two limitations from the paper. First, Redundant Memory Access and Poor Temporal Locality: naive Two-tone encoding and decoding repeatedly touch parity memory without memory-friendly scheduling. Second, Inefficient Data Traversal Pattern of Encoding: horizontal traversal has better data locality but repeated parity updates, while vertical traversal reduces parity updates but increases data reloads. So the issue is not coding correctness, but memory-access trade-offs in implementation. Fig.1 provides the encoding context.

Limitations of Existing Solutions (2/2)

- **Complex Decoding Process with Suboptimal Traversal Pattern**
- Naive decoding treats data recovery and parity re-encoding as separate operators.
- For mixed data/parity erasures, this causes repeated passes over overlapping memory regions.
- This motivates operator fusion and case-aware traversal selection.

Takeaway: the bottleneck is memory behavior in implementation, not the Two-tone coding theory itself.

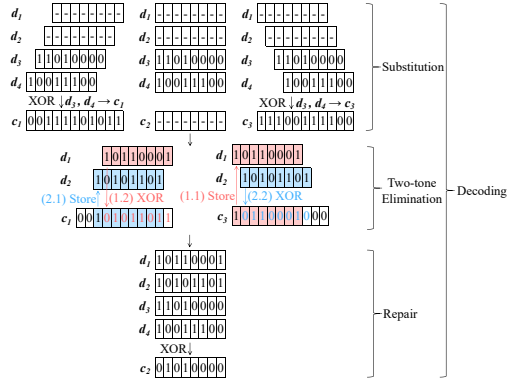
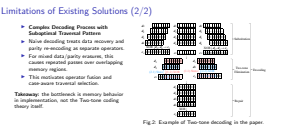


Fig.2: Example of Two-tone decoding in the paper.



The third limitation is Complex Decoding Process with Suboptimal Traversal Pattern. Naive decoding often separates data recovery and parity re-encoding into two operators. For mixed data and parity erasures, this creates repeated passes over overlapping memory regions. This is exactly where operator fusion and case-aware traversal can help. Fig.2 shows the decoding context.

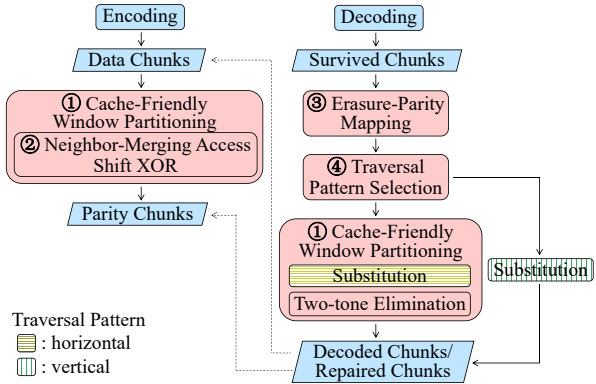
FastTT Overview

Core design principle

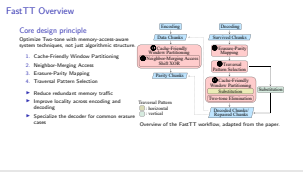
Optimize Two-tone with memory-access-aware system techniques, not just algorithmic structure.

1. Cache-Friendly Window Partitioning
2. Neighbor-Merging Access
3. Erasure-Parity Mapping
4. Traversal Pattern Selection

- Reduce redundant memory traffic
- Improve locality across encoding and decoding
- Specialize the decoder for common erasure cases



Overview of the FastTT workflow, adapted from the paper.

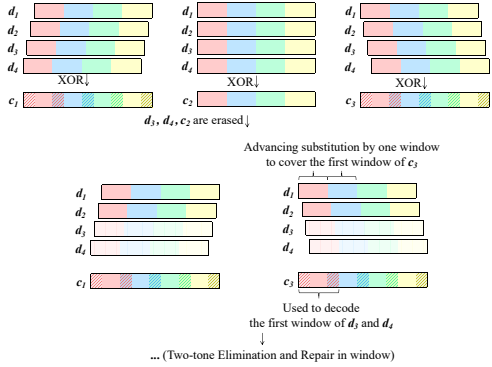


Our solution is FastTT, and the key design principle is to optimize Two-tone from a system perspective, especially memory access behavior. FastTT combines four techniques: cache-friendly window partitioning, neighbor-merging access, erasure-parity mapping, and traversal pattern selection. Together, these techniques reduce redundant memory traffic, improve locality, and adapt the decoder to common failure patterns. This is the core story of the talk.

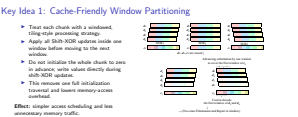
Key Idea 1: Cache-Friendly Window Partitioning

- Treat each chunk with a windowed, tiling-style processing strategy.
- Apply all Shift-XOR updates inside one window before moving to the next window.
- Do not initialize the whole chunk to zero in advance; write values directly during shift-XOR updates.
- This removes one full initialization traversal and lowers memory-access overhead.

Effect: simpler access scheduling and less unnecessary memory traffic.



Windowed processing plays a tiling-like role, improving locality without relying on hardware-specific cache sizing.



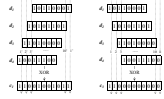
The first idea is a windowed, tiling-style processing strategy. Instead of scanning an entire chunk at once, we work window by window and complete the shift-XOR updates inside one window before moving to the next. The important implementation detail is that we do not pre-initialize the whole chunk to zero. We simply write the values directly during shift-XOR updates, which removes one full traversal and reduces memory-access overhead. The main benefit is a simpler access schedule and less unnecessary memory traffic.

Key Idea 2: Traversal Patterns and Neighbor-Merging Access

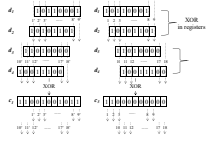
- Horizontal traversal ($\Theta((k+km)l)$): scans data chunk-by-chunk, giving strong local reads but many repeated parity writes.
- Vertical traversal ($\Theta((m+km)l)$): writes each parity symbol once, but repeatedly reloads data from different chunks.
- Neighbor-merging (FastTT, $\Theta\left((k+\frac{km}{2})l\right)$): XORs adjacent data contributions in SIMD registers before touching parity memory, cutting write traffic while preserving locality.

Design choice: keep the horizontal-style access locality, but reduce parity-memory pressure with in-register merging.

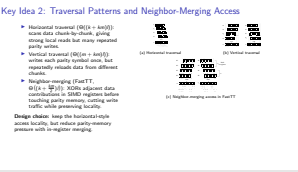
(a) Horizontal traversal



(b) Vertical traversal



(c) Neighbor-merging access in FastTT

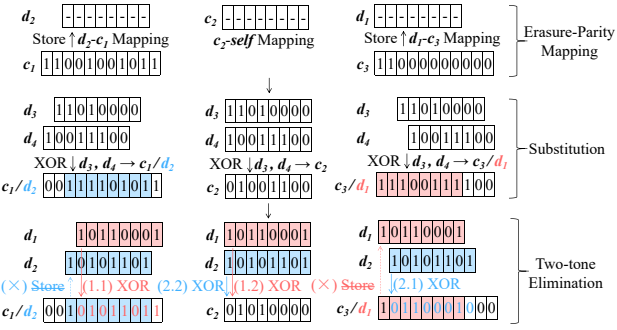


Two-tone encoding trades between two main traversal patterns with distinct memory-access behaviors. Horizontal traversal ($\Theta((k+km)l)$) scans data chunk-by-chunk and updates parity symbols per-step: this yields very good local reads but causes many parity writes. Vertical traversal ($\Theta((m+km)l)$) gathers all data symbols needed for a parity symbol and writes once: it reduces parity writes but repeatedly reloads data. FastTT's neighbor-merging reduces extra memory traffic by combining adjacent contributions inside SIMD registers before writing parity, lowering the effective cost to $\Theta\left(k+\frac{km}{2}\right)l$ and improving performance on memory-bound platforms. This vertical pattern will come back later, because FastTT reuses it as a special fast path for single-erasure decoding.

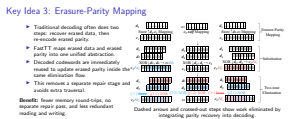
Key Idea 3: Erasure-Parity Mapping

- Traditional decoding often does two steps: recover erased data, then re-encode erased parity.
- FastTT maps erased data and erased parity into one unified abstraction.
- Decoded codewords are immediately reused to update erased parity inside the same elimination flow.
- This removes a separate repair stage and avoids extra traversal.

Benefit: fewer memory round-trips, no separate repair pass, and less redundant reading and writing.



Dashed arrows and crossed-out steps show work eliminated by integrating parity recovery into decoding.

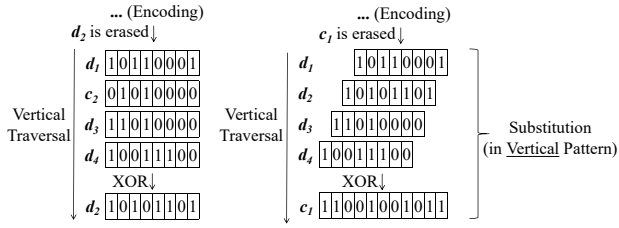


The third idea is to unify erased data and erased parity under one mapping abstraction. Traditionally, the decoder first reconstructs lost data and then runs a separate parity repair phase. FastTT removes that split. Instead, one decoding workflow handles both kinds of losses together, and decoded codewords are immediately reused to update erased parity inside the elimination process. So the key point is not that every case follows exactly the same path, but that we avoid writing results out and reading them back again for a separate repair pass. This cuts redundant memory traffic and makes the decoding workflow cleaner.

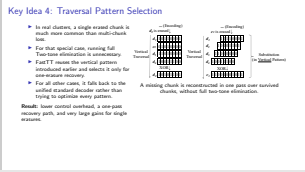
Key Idea 4: Traversal Pattern Selection

- In real clusters, a single erased chunk is much more common than multi-chunk loss.
- For that special case, running full Two-tone elimination is unnecessary.
- FastTT reuses the vertical pattern introduced earlier and selects it only for one-erasure recovery.
- For all other cases, it falls back to the unified standard decoder rather than trying to optimize every pattern.

Result: lower control overhead, a one-pass recovery path, and very large gains for single erasures.



A missing chunk is reconstructed in one pass over survived chunks, without full two-tone elimination.



The fourth idea is based on a practical observation: in real storage systems, single-erasure recovery is much more common than complicated multi-erasure recovery. This is where the earlier vertical traversal pattern comes back. FastTT is not doing a fully general smart selection for every loss pattern. Instead, when exactly one chunk is erased, it explicitly chooses the vertical pattern and reconstructs the missing chunk with a direct one-pass XOR path. For all other cases, it falls back to the unified standard decoder. That narrower fast path is enough to cut control overhead and is a major reason why single-erasure decoding speed improves so dramatically.

Flagship Results: Industrial and Academic Baselines

Industrial Baselines in Ceph

- Platform: Intel i9-14900K, AVX2, 2 MB chunks.
- Vs. ISA-L: **+59.1%** avg. encode, **+50.9%** avg. decode.
- Vs. Jerasure: **+261.4%** avg. encode, **+161.1%** avg. decode.

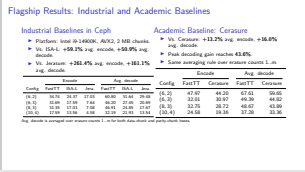
	Encode			Avg. decode		
Config	FastTT	ISA-L	Jera.	FastTT	ISA-L	Jera.
$(6, 2)$	34.74	24.37	17.03	60.80	31.64	29.48
$(6, 3)$	31.69	17.59	7.64	46.20	27.45	20.69
$(8, 3)$	31.35	17.03	7.08	46.91	24.85	17.67
$(10, 4)$	17.59	13.56	4.58	32.19	21.93	13.54

Avg. decode is averaged over erasure counts $1..m$ for both data-chunk and parity-chunk losses.

Academic Baseline: Cerasure

- Vs. Cerasure: **+13.2%** avg. encode, **+16.0%** avg. decode.
- Peak decoding gain reaches **43.6%**.
- Same averaging rule over erasure counts $1..m$.

	Encode		Avg. decode	
Config	FastTT	Cerasure	FastTT	Cerasure
$(6, 2)$	47.97	44.20	67.61	59.65
$(6, 3)$	32.01	30.97	49.39	44.82
$(8, 3)$	32.75	28.72	48.67	43.89
$(10, 4)$	24.58	19.36	37.28	33.36



Here I want to present the industrial and academic comparisons with equal weight. On the left is the comparison against the Ceph-integrated baselines, ISA-L and Jerasure, which reflects how FastTT compares with production-facing erasure coding choices. On the right is the comparison against Cerasure, which reflects the comparison with a recent academic state-of-the-art implementation. In all decoding tables here, the number is the average throughput over all erasure counts from 1 to m, including both lost data chunks and lost parity chunks. Against the Ceph baselines, FastTT improves average encoding throughput by 59.1 percent over ISA-L and 261.4 percent over Jerasure, while average decoding throughput improves by 50.9 percent and 161.1 percent respectively. Against Cerasure, FastTT still gains 13.2 percent on average in encoding and 16.0 percent on average in decoding, with a peak decoding gain of 43.6 percent.

Flagship Results: Ablation

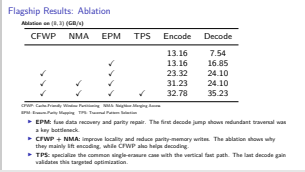
Ablation on $(8, 3)$ **(GB/s)**

CFWP	NMA	EPM	TPS	Encode	Decode
				13.16	7.54
		✓		13.16	16.85
✓		✓		23.32	24.10
✓	✓	✓		31.23	24.10
✓	✓	✓	✓	32.78	35.23

CFWP: Cache-Friendly Window Partitioning NMA: Neighbor-Merging Access

EPM: Erasure-Parity Mapping TPS: Traversal Pattern Selection

- **EPM:** fuse data recovery and parity repair. The first decode jump shows redundant traversal was a key bottleneck.
- **CFWP + NMA:** improve locality and reduce parity-memory writes. The ablation shows why they mainly lift encoding, while CFWP also helps decoding.
- **TPS:** specialize the common single-erasure case with the vertical fast path. The last decode gain validates this targeted optimization.


Flagship Results: Ablation

Ablation on $(k, 3)$ **(GB/s)**

CFWP	NMA	EPM	TPS	Encode	Decode
				13.16	7.54
		✓		13.16	16.85
✓		✓		23.32	24.10
✓	✓	✓		31.23	24.10
✓	✓	✓	✓	32.78	35.23

CFWP: Cache-Friendly Window Partitioning NMA: Neighbor-Merging Arrow
EPM: Erasure-Parity Mapping TPS: Traversal Pattern Selection

- **EPM:** fuse data recovery and parity repair. The first decode jump shows redundant traversal was a key bottleneck.
- **CFWP + NMA:** improve locality and reduce parity-memory writes. The ablation shows why they mainly lift encoding, while CFWP also helps decoding.
- **TPS:** specialize the common single-erasure case with the vertical fast path. The last decode gain validates this targeted optimization.

This slide explains why the gains happen. EPM fuses data recovery with parity repair, and the first large decode jump shows that extra traversal was a real bottleneck. CFWP and NMA then improve locality and reduce parity-memory writes, which is why they mainly lift encoding while CFWP also helps decoding. TPS gives the last decode boost by specializing the common single-erasure case with the vertical fast path.

Summary and Future Research

One-sentence summary

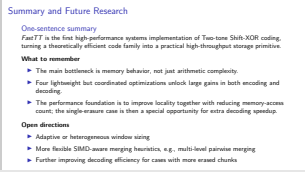
FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding, turning a theoretically efficient code family into a practical high-throughput storage primitive.

What to remember

- The main bottleneck is memory behavior, not just arithmetic complexity.
- Four lightweight but coordinated optimizations unlock large gains in both encoding and decoding.
- The performance foundation is to improve locality together with reducing memory-access count; the single-erasure case is then a special opportunity for extra decoding speedup.

Open directions

- Adaptive or heterogeneous window sizing
- More flexible SIMD-aware merging heuristics, e.g., multi-level pairwise merging
- Further improving decoding efficiency for cases with more erased chunks


Summary and Future Research

One-sentence summary

FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding, turning a theoretically efficient code family into a practical high-throughput storage primitive.

What to remember

- The main bottleneck is memory behavior, not just arithmetic complexity.
- Four lightweight but coordinated optimizations unlock large gains in both encoding and decoding.
- The performance foundation is to improve locality together with reducing memory-access count; the single-erasure case is then a special opportunity for extra decoding speedup.

Open directions

- Adaptive or heterogeneous window sizing
- More flexible SIMD-aware merging heuristics, e.g., multi-level pairwise merging
- Further improving decoding efficiency for cases with more erased chunks

To summarize in one sentence, FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding. The key lesson is that the bottleneck is memory behavior, so the real performance gains come from coordinating multiple methods that both improve locality and reduce total memory-access count. On top of that, the single-erasure case gives an extra opportunity for a specialized fast decoding path. Looking forward, there are several promising directions: adaptive window sizing, more flexible SIMD-aware merging heuristics such as multi-level pairwise merging, and continuing to improve decoding efficiency when more chunks are erased. We hope this work makes Shift-XOR coding a more practical option for real storage systems.

Thank you!

Questions?

Thank you!
Questions?

Thank you for listening. I would be happy to discuss the implementation details, the comparison with Reed-Solomon based libraries, or the implications for production storage deployments.